

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 2. Docker Essentials - Image

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate > {
```

```
public static void main(String[] args)

<servlet>
  <servlet-name>BoardController</servlet-name>
  <servlet-class>com.joneconsulting.controller.BoardCont
  <init-param>
    <param-name>user_name</param-name>
    <param-value>henmeth Lee</param-value>
  </init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명

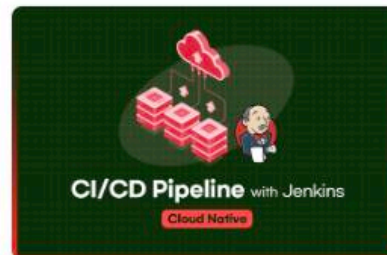


멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명



Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

수정하기

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- **Section 2: Docker Essentials - Image**
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 2.

Docker Essentials - Image

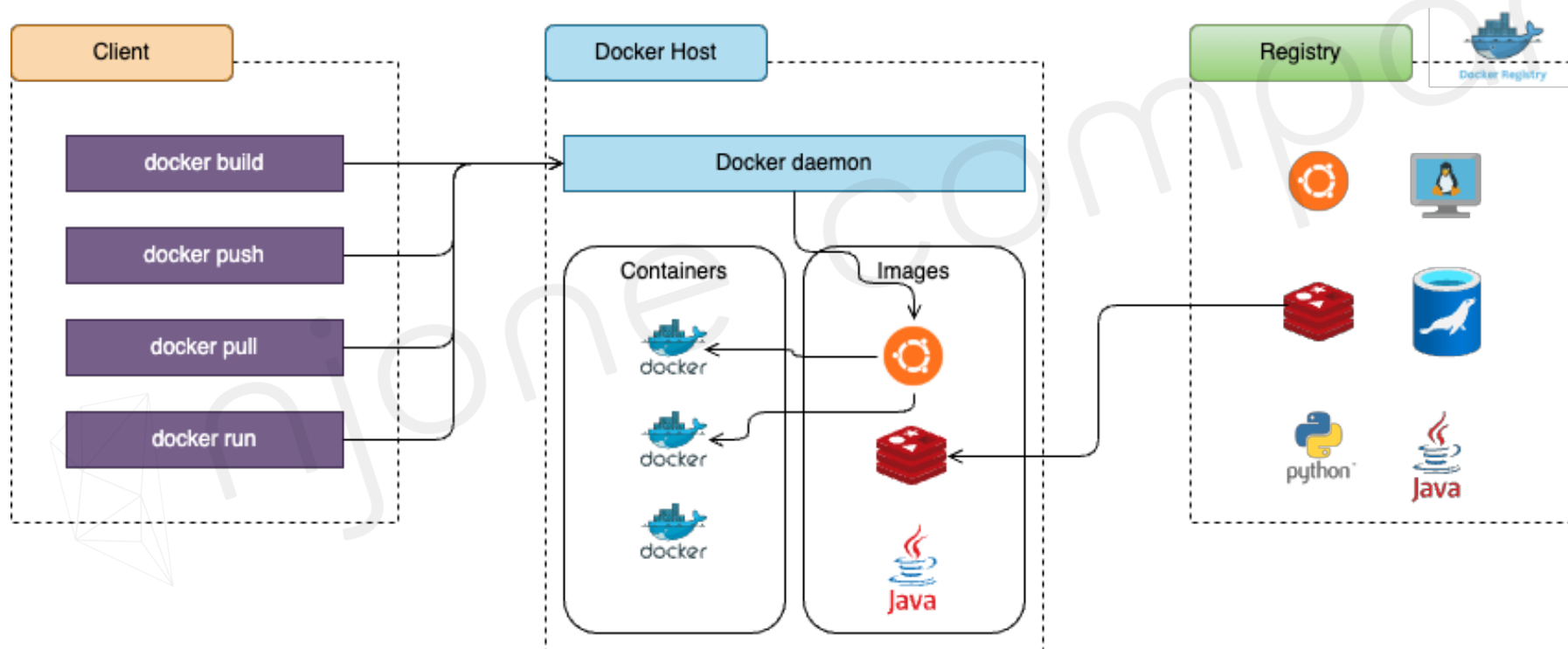
- Docker Image 작성
- Dockerfile
- 가상화 시스템 구축
- Docker Registry

Docker Image 작성

Docker Image (= Docker 실행 환경)

- 컨테이너를 만드는 데에 필요한 읽기 전용 상태의 템플릿
- 컨테이너 실행에 필요한 파일과 설정 값 등을 포함하고 있지만, 상태 값 X

| 실체화 → Container

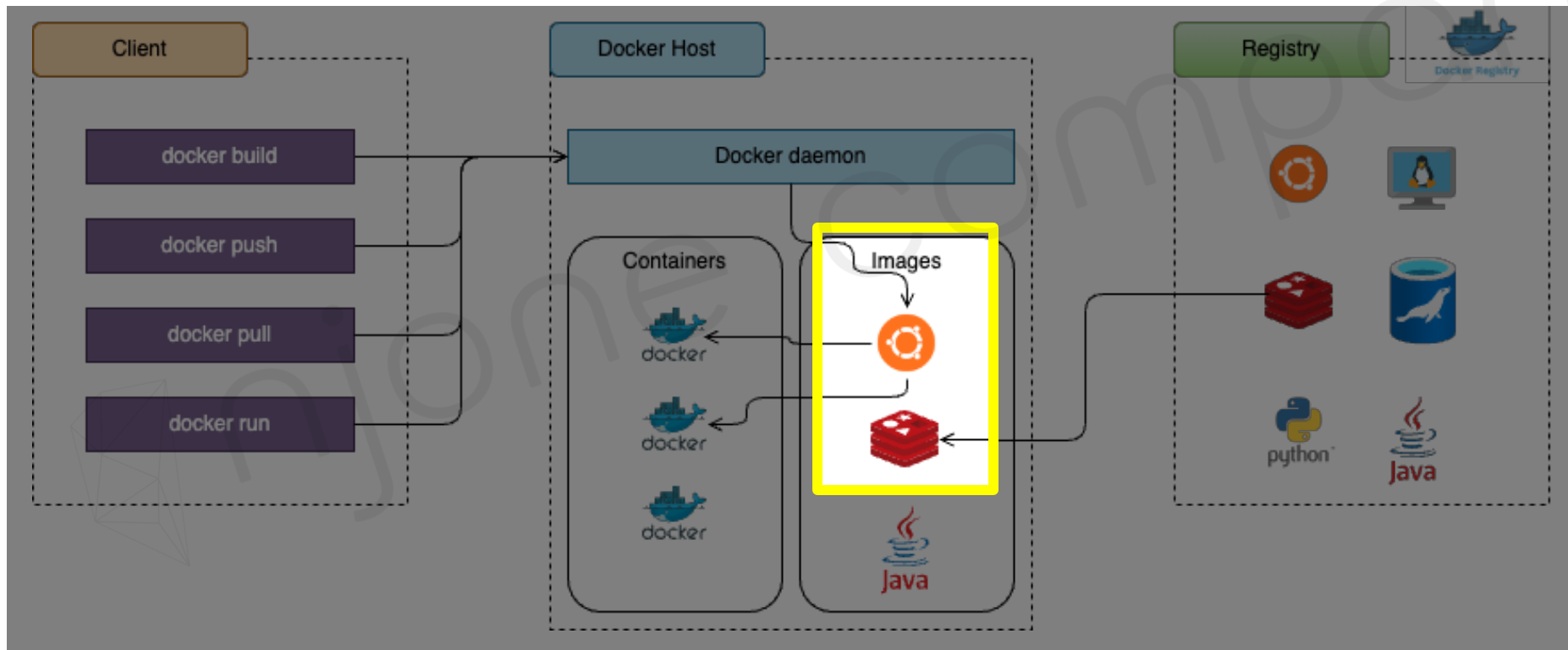


Docker Image 작성

Docker Image (= Docker 실행 환경)

- 컨테이너를 만드는 데에 필요한 읽기 전용 상태의 템플릿
- 컨테이너 실행에 필요한 파일과 설정 값 등을 포함하고 있지만, 상태 값 X

| 실체화 → Container



Docker Image 작성

Docker Image 생성 방법

- Dockerfile: 이미지 빌드용 DSL(Domain Specific Language)

```
$ touch Dockerfile
```

```
$ vi Dockerfile
```

```
FROM ubuntu:16.04
```

```
$ docker build --tag first-image:0.1 -f Dockerfile .
```

```
$ docker image ls
```

Docker Image 생성 방법

■ Dockerfile 작성 명령어

명령어	설명
FROM	Base Image 지정 명령어
RUN	특정 Layer 생성
COPY	이미지 파일 생성 시 호스트 파일 복사
ADD	이미지 파일 생성 시 호스트 파일 복사 (tar, url 가능)
WORKDIR	이미지 파일 생성 시 명령어가 실행 될 작업 디렉토리
ENTRYPOINT	컨테이너가 실행 될 때 가장 먼저 실행될 프로그램 지정 (컨테이너 실행 시 명령어 Overwrite 불가능)
CMD	컨테이너가 실행 될 때 가장 먼저 실행될 프로그램 지정 (컨테이너 실행 시 명령어 Overwrite 가능)
ENV	컨테이너 내의 환경변수 설정
EXPOSE	컨테이너의 특정 포트를 외부에 오픈

Docker 환경에서 웹 서비스 구축하기 – Node.js

- <https://github.com/joneconsulting/devops-docker/tree/section2/my-nodejs>

```
1  {
2    "name": "example",
3    "version": "1.0.0",
4    "main": "app.js",
5    "scripts": {
6      "build": "babel app.js -d dist",
7      "serve": "node app.js"
8    },
9    "dependencies": {
10     "express": "^4.17.1"
11   },
12   "devDependencies": {
13     "@babel/cli": "7.18.10",
14     "@babel/core": "7.19.1",
15     "@babel/node": "7.19.1",
16     "@babel/runtime": "7.12.5",
17     "@babel/preset-env": "7.19.1"
18   }
19 }
```

package.json

```
1  var express = require('express');
2  var app = express();
3
4  app.get('/', (req, res) => {
5    res.status(200).json({
6      message: `Hello, ${req.query.name}!`
7    });
8  });
9
10 app.listen(8000, () => {
11   console.log(`Example app for CI/CD listening on port 8000`)
12 })
```

app.js

Docker 환경에서 웹 서비스 구축하기 – Node.js

■ Docker Image 작성하기

| Step 1)

```
1 FROM alpine
2
3 RUN npm install
4
5 CMD ["npm", "start"]
```

| Step 2)

```
1 FROM node:alpine
2
3 RUN npm install
4
5 CMD ["npm", "start"]
```

| Step 3)

```
1 FROM node:alpine
2
3 COPY ./package.json ./package.json
4 COPY ./index.js ./index.js
5
6 RUN npm install
7
8 CMD ["npm", "start"]
```

| Step 4)

```
1 FROM node:alpine
2
3 WORKDIR /home/node
4
5 COPY ./package.json ./package.json
6 COPY ./index.js ./index.js
7
8 RUN npm install
9
10 CMD ["npm", "start"]
```

Docker Image 저장소란?

- Docker를 통해 생성하는 Image들을 저장해주는 저장소 (Repository)
 - | Public Registry (Docker Hub 사이트)
 - | Private Registry (기업망 내부 / 자신의 Local PC)
- Docker Image들의 위치 제어 및 CI/CD를 위한 자동화 Pipeline 구축 가능



Amazon ECR

Docker Registry

Docker Image 저장소 사용

- <https://hub.docker.com>

The image shows two overlapping screenshots of the Docker Hub website. The background screenshot displays search results for 'node', showing the 'node' repository with 1B+ downloads, 10K+ stars, and 10,644,064 pulls. The foreground screenshot shows the 'edowon0623' repository page, listing three repositories: 'first-service', 'cicd-project', and 'ansible'.

Background Screenshot (Search Results for 'node'):

- Search bar: node
- Filters: Products (Images, Extensions, Plugins), Trusted Content (Docker Official Image, Verified Publisher, Sponsored OSS), Operating Systems (Linux).
- Results: 1 - 25 of 10,000 results for node.
- Repository: node (Updated 3 days ago). Description: Node.js is a JavaScript-based platform for server-side and networking applications. Pulls: 10,644,064 (Last week).

Foreground Screenshot (Repository Page for 'edowon0623'):

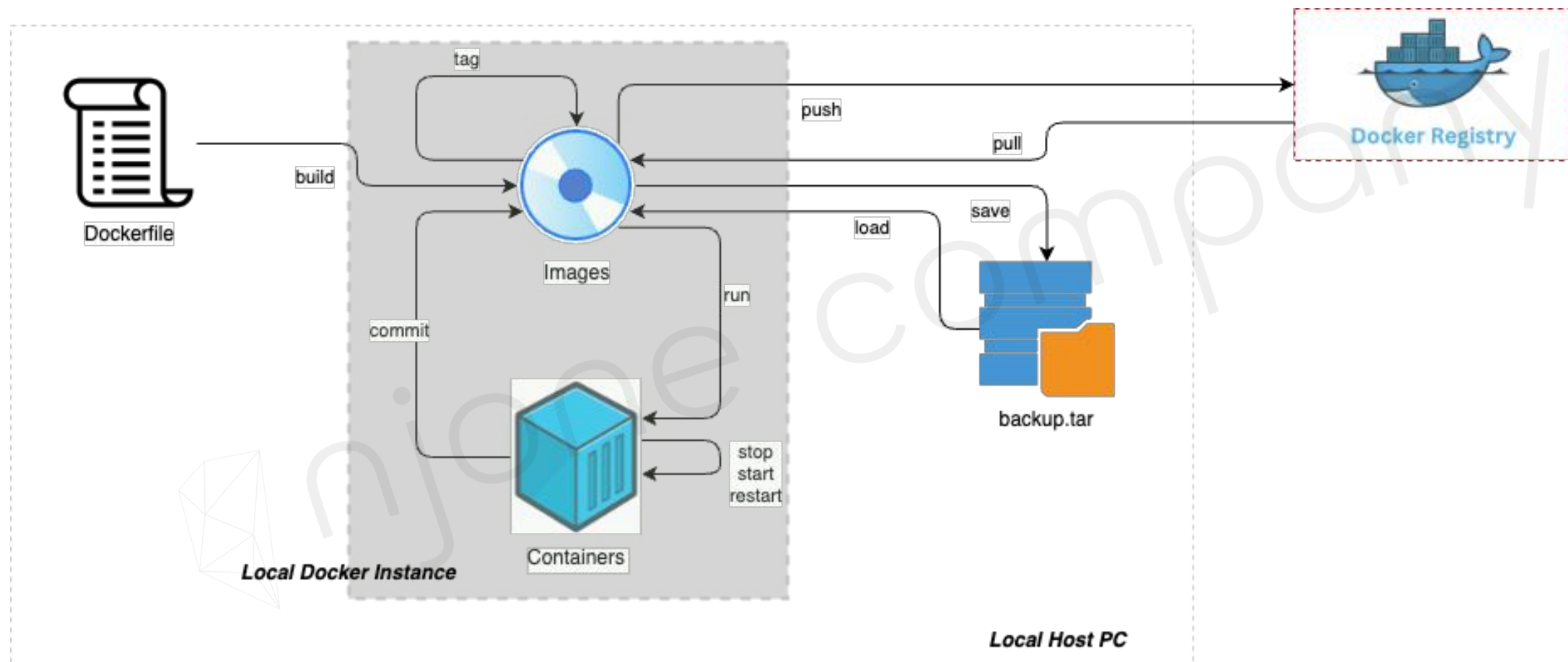
- Search bar: node
- Repository: edowon0623
- Search by repository name: Search by repository name
- Filter: All Content
- Create repository button
- Repositories listed:

Repository	Status	Stars	Downloads	Visibility
edowon0623 / first-service	Inactive	0	19	Public
edowon0623 / cicd-project	Inactive	0	22	Public
edowon0623 / ansible	Inactive	3	744	Public

Docker Registry

Docker Image 저장소 사용

- Docker Image의 생성과 사용 Flow



Docker Registry

CI/CD 자동화 파이프라인

Continuous Integration (CI)

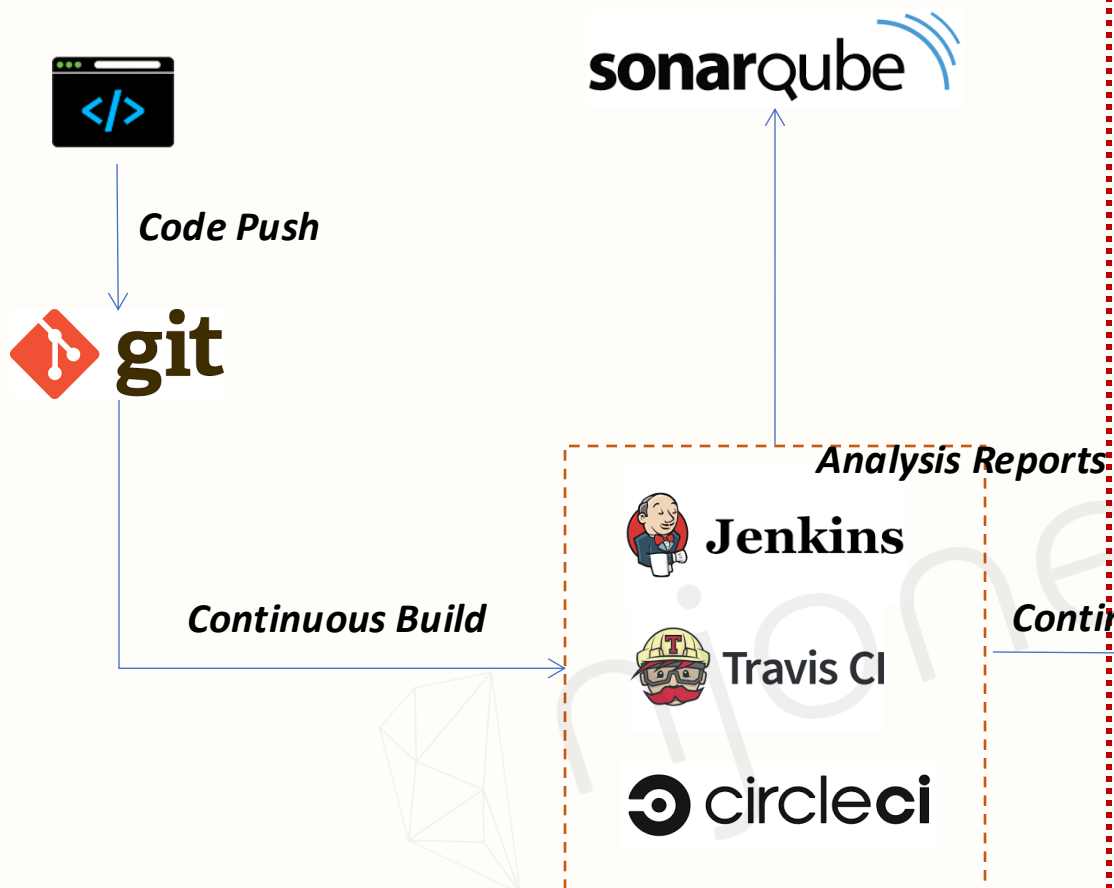
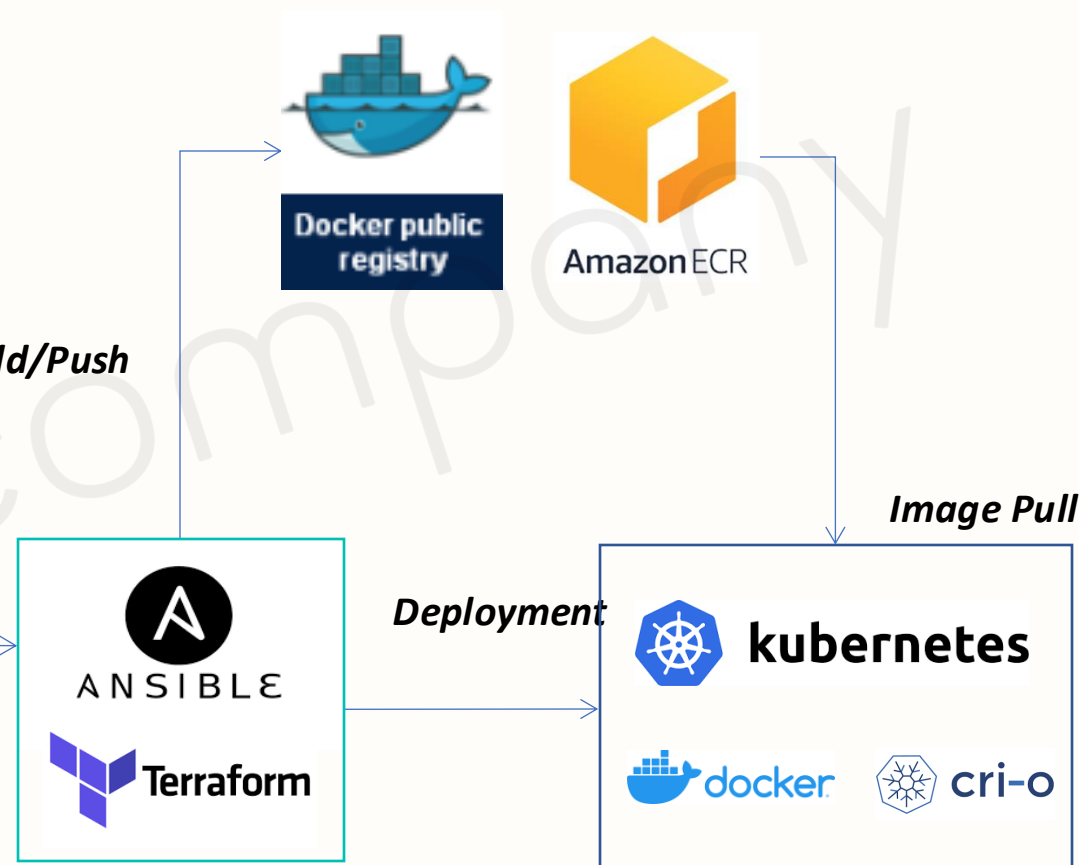


Image Build/Push



Continuous Delivery (CD)

Local Registry 사용

- Registry Container 사용

| <https://hub.docker.com/search?q=registry>

```
$ docker run -d -p 5000:5000 --restart always --name registry registry:2
```

```
$ docker pull ubuntu
```

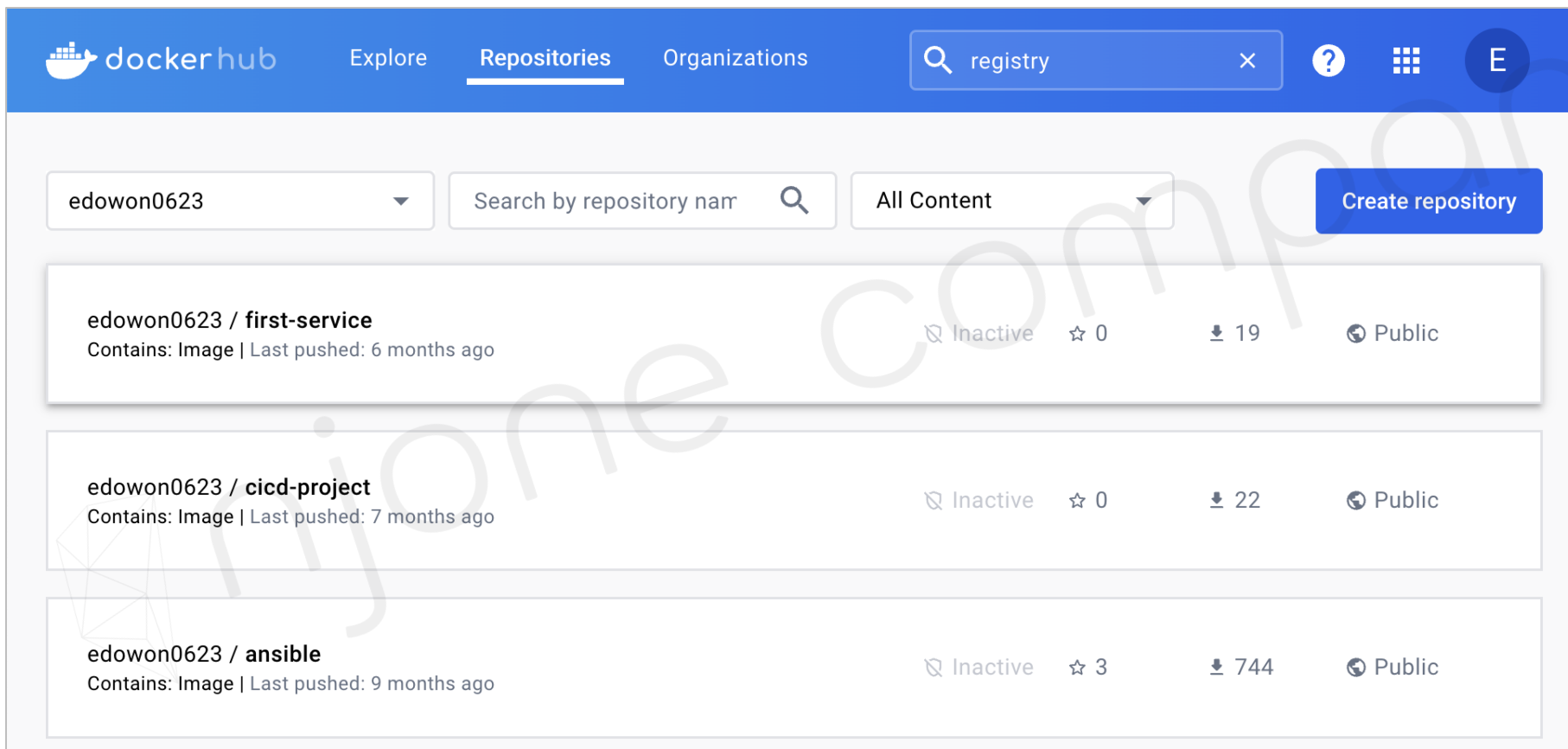
```
$ docker tag ubuntu localhost:5000/ubuntu
```

```
$ docker push localhost:5000/ubuntu
```

Cloud Registry 사용

- Docker Hub 사이트

<https://hub.docker.com>



The screenshot shows the Docker Hub interface. The top navigation bar is blue with the Docker Hub logo, 'Explore', 'Repositories' (selected), 'Organizations', a search bar containing 'registry', a help icon, a grid icon, and a user profile icon 'E'. Below the navigation bar, there's a search section with a dropdown menu showing 'edowon0623', a search input field with the placeholder 'Search by repository name', a dropdown menu showing 'All Content', and a 'Create repository' button. The main content area displays a list of repositories for the user 'edowon0623'. Each repository entry includes the repository name, a description, the last pushed time, and status icons for 'Inactive', stars, downloads, and 'Public'.

Repository Name	Description	Last Pushed	Status	Stars	Downloads	Visibility
edowon0623 / first-service	Contains: Image	6 months ago	Inactive	0	19	Public
edowon0623 / cicd-project	Contains: Image	7 months ago	Inactive	0	22	Public
edowon0623 / ansible	Contains: Image	9 months ago	Inactive	3	744	Public